

JavaScript Complete Handbook

Part 1: Fundamentals & Core Language

Absolute Beginner to Strong Intermediate · Interview Ready · 175+ Q&A

Chapters covered: Introduction · Variables & Data Types · Operators · Type Coercion · Control Flow · Functions · Scope · Hoisting · Arrays · Strings · Objects · 175+ Interview Q&A · Cheat Sheets

Table of Contents

Chapter 01	Introduction to JavaScript
Chapter 02	Variables & Data Types
Chapter 03	Operators
Chapter 04	Type Conversion & Coercion
Chapter 05	Control Flow
Chapter 06	Functions
Chapter 07	Scope
Chapter 08	Hoisting
Chapter 09	Arrays
Chapter 10	Strings
Chapter 11	Objects
Chapter 12	Interview Questions (175+)
Chapter 13	Cheat Sheet & Quick Revision

Introduction to JavaScript

1.1 What Is JavaScript?

JavaScript is a high-level, interpreted, dynamically-typed, single-threaded, prototype-based programming language. It is the only language natively supported by all major web browsers, making it the core language of the web. Today JavaScript runs everywhere: browsers, servers (Node.js), mobile apps (React Native), desktop apps (Electron), IoT devices, and databases.

1.2 History

1995	Brendan Eich at Netscape creates 'Mocha' in 10 days. Renamed LiveScript, then JavaScript for marketing.
1996	Microsoft creates JScript for IE. Competing implementations cause fragmentation.
1997	JavaScript submitted to ECMA International. First ECMAScript standard (ES1) published.
1999	ES3 — regular expressions, try/catch, string methods.
2009	ES5 — strict mode, JSON support, Array methods (forEach, map, filter, reduce). Node.js by Ryan Dahl.
2015	ES6/ES2015 — let/const, arrow functions, classes, Promises, modules, destructuring, template literals.
2016+	Yearly releases: ES2016–ES2024 add async/await, optional chaining, nullish coalescing, private fields.

1.3 ECMAScript vs JavaScript

ECMAScript is the official specification maintained by ECMA International (TC39 committee). JavaScript is the implementation — plus browser/runtime-specific APIs (DOM, fetch, setTimeout). Think of ECMAScript as the recipe and JavaScript as the dish.

1.4 JavaScript Engines

Engine	Used By	Key Feature
V8	Chrome, Node.js, Edge	JIT, TurboFan optimising compiler
SpiderMonkey	Firefox	First JS engine, by Mozilla
JavaScriptCore	Safari, React Native	Nitro optimiser
Hermes	React Native	Optimised for mobile startup

1.5 JIT Compilation

JavaScript uses Just-In-Time (JIT) Compilation. The engine compiles frequently-executed (hot) code paths to native machine code at runtime, while interpreting cold code immediately. This gives performance close to compiled languages.

Pipeline: Source → Parser → AST → Interpreter (Ignition) → Bytecode. Hot paths → JIT Compiler (TurboFan) → Optimised Machine Code → Native CPU Execution.

1.6 Browser vs Node.js

Feature	Browser	Node.js
Global object	window	global / globalThis
DOM access	Yes	No
File system	No (sandboxed)	Yes (fs module)
Modules	ES Modules	CommonJS + ES Modules
Use case	UI, user interaction	APIs, CLI tools, servers

1.7 Interview Questions

Q: What is the difference between JavaScript and ECMAScript?

ECMAScript is the specification/standard. JavaScript is the implementation. ECMA International's TC39 committee defines ECMAScript. JavaScript engines (V8, SpiderMonkey) implement it. JavaScript also includes runtime-specific APIs (DOM in browsers, fs in Node.js) that are not part of ECMAScript.

Q: Why is JavaScript called a single-threaded language?

JavaScript runs on a single call stack — only one piece of code executes at a time. Non-blocking behaviour is achieved through the event loop, which offloads I/O to the browser/OS while the main thread stays free.

Q: What is JIT compilation and how does it benefit JavaScript?

JIT (Just-In-Time) compilation converts hot JavaScript code to native machine code at runtime. The engine identifies frequently-executed code (hot spots) and compiles them for speed while interpreting cold code immediately. This gives JavaScript near-native performance.

Variables & Data Types

2.1 Variable Declaration: var, let, const

Feature	var	let	const
Scope	Function	Block	Block
Hoisted	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Global property	Yes (window.x)	No	No
When to use	Legacy only	Mutable values	Constants/refs

```
// var – function scoped, avoid in modern code
var x = 10;
var x = 20;    // redeclaration: ALLOWED (confusing!)
x = 30;        // reassignment: allowed

// let – block scoped, use for mutable values
let count = 0;
// let count = 1; // SyntaxError: already declared
count = 1;     // reassignment: allowed

// const – block scoped; reference is constant, NOT the value
const PI = 3.14159;
// PI = 3;      // TypeError: assignment to constant

const user = { name: 'Alice' };
user.name = 'Bob'; // ALLOWED – mutating the object
// user = {};     // TypeError – can't reassign the reference
```

2.2 The 8 Data Types

Type	Description	Examples
Number	64-bit IEEE 754 float. Integers and decimals. Special values: Infinity, -Infinity, NaN.	42, 3.14, NaN, Infinity
String	Sequence of UTF-16 chars. Immutable.	'hello', `Hi \${name}`
Boolean	Logical values. Result of comparisons.	true, false
Null	Intentional absence of value. <code>typeof null === 'object'</code> (historical bug).	null
Undefined	Variable declared but not assigned.	undefined
Symbol	Unique, immutable identifier.	Symbol('id')
BigInt	Arbitrary precision integers.	9007199254740992n
Object	Key-value pairs. Arrays, functions are objects.	{a:1}, [1,2], function(){}

2.3 typeof Operator

```
typeof 42           // 'number'
typeof 'hello'      // 'string'
typeof true         // 'boolean'
typeof undefined    // 'undefined'
typeof null         // 'object' ← BUG! (legacy)
typeof {}           // 'object'
typeof []           // 'object' ← arrays are objects
typeof function(){} // 'function' ← special case
typeof Symbol()     // 'symbol'
typeof 42n          // 'bigint'

null === null       // true — only reliable null check
Array.isArray([])   // true — use this for arrays
```

2.4 Common Mistakes & Best Practices

- ✗ Using var instead of let/const — leads to scope confusion
- ✗ Expecting typeof null to return 'null' — it returns 'object'
- ✗ Using == to check null — null == undefined is true; use ===
- ✗ Mutating a const object and expecting an error — const only prevents reassignment
- ✗ Confusing undefined (not assigned) with null (deliberately empty)
- ✓ Default to const — only use let when you need to reassign
- ✓ Never use var in modern JavaScript
- ✓ Use Array.isArray() to check for arrays, not typeof
- ✓ Use === null to check for null specifically
- ✓ Use Number.isNaN(), not isNaN() — global isNaN coerces first

2.5 Interview Questions

Q: What is the difference between var, let, and const?

var: function-scoped, hoisted as undefined, can be redeclared, attaches to global object. let: block-scoped, hoisted to TDZ, cannot be redeclared. const: block-scoped, hoisted to TDZ, cannot be reassigned (object contents can be mutated). Use const by default, let for mutation, never var.

Q: Why does typeof null return 'object'?

Historical bug from JavaScript's first implementation (1995). Values were stored as type tags; null's tag was 0, same as objects. The bug was never fixed to preserve backwards compatibility. Always check for null with === null.

Operators

3.1 Arithmetic Operators

```

5 + 3    // 8    addition
5 - 3    // 2    subtraction
5 * 3    // 15   multiplication
5 / 3    // 1.666 division (always float)
5 % 3    // 2    modulo (remainder)
5 ** 3   // 125  exponentiation (ES2016)

// Increment / Decrement
let x = 5;
console.log(x++); // 5 (uses x, then increments → 6)
console.log(++x); // 7 (increments to 7, then uses)

// Unary + for number conversion
+'42'      // 42
+true      // 1
+undefined // NaN

```

3.2 Comparison Operators

```

// Strict equality (===) – no type coercion
5 === 5    // true
5 === '5'  // false (different types)

// Loose equality (==) – type coercion happens
5 == '5'   // true (string coerced to number)
null == undefined // true (special rule)
0 == false // true (false coerced to 0)
'' == false // true (both coerce to 0)

// String comparison (lexicographic)
'banana' > 'apple' // true (b > a)
'10' > '9'         // false! ('1' < '9' as strings)
10 > 9             // true  (use numbers, not strings)

```

3.3 Logical Operators

```

// && (AND) – returns first falsy or last value
true && 'hello'    // 'hello'
false && 'hello'   // false  (short-circuits)
null && doWork()   // null   (doWork never called)

// || (OR) – returns first truthy or last value
false || 'default' // 'default'
0 || 42            // 42    (0 is falsy)

// ?? (Nullish Coalescing – ES2020) – only null/undefined
null ?? 'default'  // 'default'

```

```

0 ?? 'default'    // 0    (0 is NOT nullish!)
'' ?? 'default'   // ''   (empty string is NOT nullish!)

// ! (NOT)
!true    // false
!0       // true
!!value  // double negation → convert to boolean

```

3.4 Assignment Operators

```

let x = 10;
x += 5;    // 15    x = x + 5
x -= 3;    // 12    x = x - 3
x *= 2;    // 24    x = x * 2
x /= 4;    // 6     x = x / 4
x %= 4;    // 2     x = x % 4
x **= 3;   // 8     x = x ** 3

// Logical assignment (ES2021)
x ||= 5;   // assign if x is falsy
x &&= 5;   // assign if x is truthy
x ??= 5;   // assign if x is null/undefined

```

3.5 Ternary & Optional Chaining

```

// Ternary: condition ? ifTrue : ifFalse
const status = age >= 18 ? 'adult' : 'minor';

// Optional chaining (?.) – ES2020
const user = { address: { city: 'London' } };
user?.address?.city    // 'London'
user?.phone?.number    // undefined (no throw!)
user?.getName?.()      // calls if exists, else undefined
arr?.[0]               // safe array access

```

3.6 Interview Questions

Q: What is the difference between == and ===?

=== (strict equality) checks value AND type without coercion. == (loose equality) performs type coercion before comparing. '5' == 5 is true but '5' === 5 is false. Always use === to avoid subtle bugs.

Q: What is the difference between || and ??

|| returns first truthy value — treats 0, "", false, null, undefined as falsy. ?? returns first non-nullish value — only treats null and undefined specially, preserving 0 and empty string. Use ?? when 0 or "" are valid values.

Type Conversion & Coercion

4.1 Explicit Conversion

```
// To Number
Number('42')      // 42
Number('')        // 0
Number(null)       // 0
Number(undefined) // NaN
Number(true)       // 1
Number(false)      // 0
Number('abc')      // NaN
parseInt('42px')   // 42 (stops at non-digit)
parseFloat('3.14abc') // 3.14
```

```
// To String
String(42)        // '42'
String(null)       // 'null'
String(undefined) // 'undefined'
(255).toString(16) // 'ff' (hex)
```

// To Boolean — ONLY these 7 values are falsy:

```
Boolean(0)        // false
Boolean('')       // false
Boolean(null)     // false
Boolean(undefined) // false
Boolean(NaN)      // false
Boolean(false)    // false
// Everything else is true:
Boolean('0')      // true  (non-empty string!)
Boolean([])       // true  (empty array!)
Boolean({})       // true  (empty object!)
```

4.2 Implicit Coercion — The Famous Traps

```
// + prefers string concatenation
'5' + 3          // '53'
1 + '2' + 3      // '123' (left-to-right: '12' + 3 = '123')
1 + 2 + '3'      // '33'  (left-to-right: 3 + '3' = '33')
```

// Arithmetic (-, *, /) forces number

```
'5' - 3          // 2
'5' * '2'         // 10
true + true       // 2
```

// Tricky comparisons

```
null > 0         // false
null == 0        // false
null >= 0        // true  (inconsistent!)
NaN == NaN       // false (NaN is not equal to itself!)
```

4.3 The 7 Falsy Values

```
// Memorise these – everything else is truthy
false, 0, -0, 0n, '', null, undefined, NaN

// TRUTHY traps – these surprise people:
'0'    // truthy (non-empty string)
[]     // truthy (empty array)
{}     // truthy (empty object)
-1     // truthy (non-zero number)
```

4.4 Interview Questions

Q: What is the output of `[] + {}` and `{ } + []`?

`[] + {} = '[object Object]'`: `[]` converts to `"` (empty string), `{ }` converts to `'[object Object]'`, string concat. `{ } + [] = 0`: when `{ }` starts a statement, JS parses it as an empty block. Then `+[]` is unary `+` applied to `[]` which converts `"` to `0`.

Q: Why is `NaN !== NaN`?

`NaN` (Not a Number) is the only value in JavaScript not equal to itself. This is per IEEE 754 specification. Use `Number.isNaN()` to check for `NaN`.

Q: What is the difference between `null` and `undefined` in type coercion?

`null` and `undefined` are loosely equal (`null == undefined` is `true`) but both are strictly unequal to other values. In numeric context: `Number(null) = 0` but `Number(undefined) = NaN`. So `null >= 0` is `true` (`null` coerces to `0`) but `null == 0` is `false` (`null` only equals `null/undefined`).

Control Flow

5.1 if / else if / else

```
const score = 75;
if (score >= 90) {
  console.log('A');
} else if (score >= 80) {
  console.log('B');
} else if (score >= 70) {
  console.log('C');
} else {
  console.log('F');
} // Output: 'C'
```

5.2 switch

```
const day = 'Monday';
switch (day) {
  case 'Monday':
  case 'Tuesday': // fall-through: both match this block
    console.log('Weekday');
    break; // MUST have break to prevent fall-through!
  case 'Saturday':
  case 'Sunday':
    console.log('Weekend');
    break;
  default:
    console.log('Unknown');
}
// switch uses strict equality (===)
```

5.3 for Loops

```
// Classic for loop
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}

// for...of – iterate over iterable VALUES (arrays, strings)
const fruits = ['apple', 'banana', 'cherry'];
for (const fruit of fruits) {
  console.log(fruit);
}

// for...in – iterate over object KEYS
const obj = { a: 1, b: 2, c: 3 };
for (const key in obj) {
  console.log(key, obj[key]); // 'a' 1, 'b' 2, 'c' 3
}
// WARNING: for...in also iterates prototype properties!
```

5.4 while & do-while

```
// while – checks condition BEFORE each iteration
let count = 0;
while (count < 3) {
  console.log(count); // 0, 1, 2
  count++;
}

// do-while – executes AT LEAST ONCE, checks AFTER
let n = 10;
do {
  console.log(n); // prints 10 even though 10 >= 5
  n--;
} while (n < 5);
```

5.5 break & continue

```
// break – exits the loop entirely
for (let i = 0; i < 10; i++) {
  if (i === 5) break;
  console.log(i); // 0, 1, 2, 3, 4
}

// continue – skips current iteration
for (let i = 0; i < 5; i++) {
  if (i === 2) continue;
  console.log(i); // 0, 1, 3, 4 (skips 2)
}

// Labelled break – break out of nested loops
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) break outer;
    console.log(i, j); // only: 0 0
  }
}
```

5.6 Interview Questions

Q: What is the difference between `for...in` and `for...of`?

`for...in` iterates over enumerable property KEYS of an object (including inherited ones). `for...of` iterates over VALUES of any iterable (arrays, strings, Maps, Sets). Don't use `for...in` on arrays — it iterates indices as strings and may include prototype properties.

Q: When would you use `do-while` instead of `while`?

When you need the loop body to execute at least once regardless of the condition. Common use case: prompting a user for valid input — always show the prompt once, then keep prompting if input is invalid.

Functions

6.1 Function Declaration

```
// Fully hoisted – can call BEFORE the declaration
greet('Alice'); // 'Hello, Alice!'

function greet(name) {
  return `Hello, ${name}!`;
}

// Default parameters (ES6)
function add(a, b = 0) { return a + b; }
add(5); // 5 (b defaults to 0)
add(5, 3); // 8

// Rest parameters
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10
```

6.2 Function Expression

```
// NOT fully hoisted – variable is undefined/TDZ before this line
const multiply = function(x, y) {
  return x * y;
};
multiply(4, 5); // 20

// Named function expression (useful for recursion/stack traces)
const factorial = function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1); // 'fact' usable inside
};
factorial(5); // 120
```

6.3 Arrow Functions

```
// Concise syntax
const double = x => x * 2; // single param, implicit return
const add = (a, b) => a + b; // multiple params
const greet = () => 'Hello!'; // no params

// Block body (needs explicit return)
const square = x => {
  const result = x * x;
  return result;
};

// Returning an object literal – wrap in parentheses!
const makeUser = name => ({ name, role: 'user' });
```

```
// Without parens: name => { name } is a BLOCK, not an object!

// Key differences from regular functions:
// - No own 'this' (uses enclosing this)
// - No 'arguments' object (use rest: ...args)
// - Cannot be used as constructors (no new)
// - No prototype property
```

6.4 IIFE — Immediately Invoked Function Expression

```
// Define and immediately call
(function() {
  const secret = 'private'; // not accessible outside
  console.log('IIFE runs immediately');
})();

// Arrow IIFE
(() => { console.log('Arrow IIFE'); })();

// Async IIFE for top-level await
(async () => {
  const data = await fetchData();
  console.log(data);
})();
```

6.5 First-Class Functions

```
// Functions as arguments (callback)
function runTwice(fn) { fn(); fn(); }
runTwice(() => console.log('Hi')); // Hi Hi

// Functions as return values (closure)
function makeAdder(x) {
  return function(y) { return x + y; };
}
const add5 = makeAdder(5);
add5(3); // 8
add5(7); // 12
```

6.6 Interview Questions

Q: What is the difference between function declaration and function expression?

Function declaration: `function foo() {}` — fully hoisted, can call before. Function expression: `const foo = function() {}` — only variable hoisted, not function (undefined/TDZ). Arrow: `const foo = () => {}` — expression with no own `this`/arguments. Declarations suit utility functions; expressions suit passing around or conditional creation.

Q: What is a higher-order function?

A function that takes another function as an argument, returns a function, or both. Examples: `Array.map/filter/reduce` take callback functions. `setTimeout/setInterval` take callbacks. Functions like `memoize`, `debounce`, and `curry` return functions. Possible because functions are first-class objects in JavaScript.

Scope

7.1 What Is Scope?

Scope defines the accessibility of variables — where in your code a variable can be read or written. JavaScript uses lexical scoping: scope is determined by where code is written, not where it is executed.

7.2 Global Scope

```
var globalVar = 'global';
let globalLet = 'also global';

function test() {
  console.log(globalVar); // accessible
  console.log(globalLet); // accessible
}

// var at global level attaches to window (browser)
var name = 'Alice';
console.log(window.name); // 'Alice'

// let/const do NOT attach to window
let age = 25;
console.log(window.age); // undefined
```

7.3 Function Scope

```
function test() {
  if (true) {
    var leaked = 'I leaked!'; // scoped to test(), NOT if block
    let blocked = 'I stayed!'; // scoped to the if block
  }
  console.log(leaked); // 'I leaked!'
  // console.log(blocked); // ReferenceError
}
```

7.4 Block Scope & the Classic Loop Bug

```
// let – each iteration gets its OWN binding
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2 ✓
}

// var – all iterations SHARE the same variable
for (var j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 100); // 3, 3, 3 ✗
}
```

7.5 Scope Chain

```
function outer() {
  let x = 10;
```

```

function middle() {
  let y = 20;
  function inner() {
    let z = 30;
    console.log(x + y + z); // 60 – walks scope chain
  }
  inner();
}
middle();
}

// Scope chain for inner():
// inner scope { z:30 } → middle scope { y:20 }
//   → outer scope { x:10 } → global scope → null (end)
// Variable lookup goes UP the chain until found (or ReferenceError)

```

7.6 Interview Questions

Q: What is the scope chain?

When a variable is accessed, JavaScript looks in the current scope first. If not found, it moves to the outer scope, then outer's outer, up to global. If not found anywhere, `ReferenceError` is thrown. This chain is determined at definition time (lexical scoping), not call time.

Q: Why does `var` leak out of if blocks but not function bodies?

`var` is function-scoped, not block-scoped. Blocks (`if`, `for`, `while`, `{}`) do not create a new scope for `var` — only functions do. This is a design flaw in early JavaScript. `let` and `const` are block-scoped and solve this problem.

Hoisting

8.1 What Is Hoisting?

During the creation phase of an Execution Context, the JavaScript engine scans code and allocates memory for all variable and function declarations before execution begins. Only declarations are hoisted, NOT initializations.

8.2 Variable Hoisting

```
// What you write:
console.log(x); // undefined (not ReferenceError!)
var x = 5;
console.log(x); // 5

// How the engine sees it:
var x;           // declaration hoisted (value = undefined)
console.log(x);  // undefined
x = 5;           // assignment stays in place

// let and const – hoisted but in Temporal Dead Zone (TDZ)
// console.log(y); // ReferenceError: Cannot access 'y' before init
let y = 10;
console.log(y);  // 10
```

8.3 Function Hoisting

```
// Function DECLARATIONS are fully hoisted (with body)
greet(); // 'Hello!' – works before declaration
function greet() { console.log('Hello!'); }
```

```
// Function EXPRESSIONS are NOT fully hoisted
// sayHi(); // TypeError: sayHi is not a function
var sayHi = function() { console.log('Hi!'); };
```

```
// Priority: function declarations win over var
console.log(typeof foo); // 'function' (not 'undefined')
var foo = 'string';
function foo() {}
console.log(typeof foo); // 'string' (assignment happened after)
```

8.4 Temporal Dead Zone (TDZ)

The TDZ is the period between the start of a scope and the point where a `let/const` variable is initialised. Accessing the variable during this period throws a `ReferenceError`. The TDZ exists to catch use-before-declare bugs that `var` silently allowed.

```
function example() {
  // TDZ starts here for x
  console.log(x); // ReferenceError: Cannot access 'x' before init
  let x = 10;     // TDZ ends here – x is now initialised
}
```

```
    console.log(x);  // 10  
}
```

8.5 Interview Questions

Q: What is hoisting and how does it differ for var vs let/const?

Hoisting moves declarations to top of scope during creation phase. `var`: hoisted AND initialised to `undefined` — accessible before declaration (value is `undefined`). `let/const`: hoisted but NOT initialised — placed in TDZ. Accessing before declaration throws `ReferenceError`. Function declarations: fully hoisted with body.

Q: What is the Temporal Dead Zone?

The TDZ is the period from the start of a block scope to the point where a `let/const` declaration is reached. During this period, the variable exists (hoisted) but accessing it throws `ReferenceError`. This prevents the confusing pattern `var` allowed where a variable could be accessed as `undefined` before its declaration.

Arrays

9.1 Creating Arrays

```
const arr1 = [1, 2, 3];           // literal (preferred)
const arr2 = new Array(3);        // [empty × 3]
const arr3 = Array.from('abc');   // ['a', 'b', 'c']
const arr4 = Array.from({length:3}, (_,i) => i*2); // [0,2,4]
const arr5 = [...arr1, ...arr3];   // [1,2,3,'a','b','c']
```

9.2 Core Methods

Method	Mutates?	Returns	Notes
map(fn)	No	New array	Transform each element. Same length as original.
filter(fn)	No	New array	Keep only elements where callback returns true.
reduce(fn, init)	No	Any type	Accumulate all elements into a single value.
find(fn)	No	First match / undefined	First element matching condition.
findIndex(fn)	No	Index or -1	Index of first matching element.
some(fn)	No	boolean	true if at least ONE element passes. Short-circuits.
every(fn)	No	boolean	true if ALL elements pass. Short-circuits.
sort(fn)	YES	Same array	Sorts in place. Always provide compareFn for numbers!
slice(s,e)	No	New array	Copy sub-array. end is exclusive. Supports negatives.
splice(s,d,...i)	YES	Removed items	Remove/insert elements. Returns removed.
flat(depth)	No	New array	Flatten nested arrays to given depth.
flatMap(fn)	No	New array	map() then flat(1) in one pass.
push(...items)	YES	New length	Add to end.
pop()	YES	Removed item	Remove from end.
unshift(...items)	YES	New length	Add to start.
shift()	YES	Removed item	Remove from start.
includes(v)	No	boolean	Value exists? Handles NaN correctly.
indexOf(v)	No	Index or -1	First index. Does NOT handle NaN.
forEach(fn)	No	undefined	Side effects only. No break. No return value.
join(sep)	No	String	Array to string with separator.

9.3 Code Examples

```
const nums = [1, 2, 3, 4, 5];

// map – transform
nums.map(n => n * 2)           // [2,4,6,8,10]

// filter – keep matching
```

```

nums.filter(n => n % 2 === 0) // [2,4]

// reduce – accumulate
nums.reduce((acc, n) => acc + n, 0) // 15

// sort numbers (always provide compareFn!)
[10, 1, 21].sort() // [1,10,21] WRONG (lexicographic)
[10, 1, 21].sort((a,b) => a-b) // [1,10,21] CORRECT ascending

// slice vs splice
const arr = [1,2,3,4,5];
arr.slice(1, 3) // [2,3] – does NOT mutate
arr.splice(1, 2) // returns [2,3]; arr is now [1,4,5] – MUTATES

```

9.4 Interview Questions

Q: What is the difference between map, filter, and reduce?

map: transforms each element, returns same-length array. filter: keeps elements matching condition, returns shorter/equal array. reduce: aggregates all elements into a single value (any type). All three return new arrays/values and don't mutate the original.

Q: What is the difference between slice and splice?

slice(start, end): non-mutating, returns new array from start to end (exclusive). splice(start, deleteCount, ...items): mutates original array, removes deleteCount elements, optionally inserts new ones, returns array of removed elements.

Strings

10.1 Creating Strings

```
const s1 = 'single quotes';
const s2 = "double quotes";
const s3 = `template literal ${1 + 2}`; // '3'

// Multi-line template literal
const html = `
  <div>
    <h1>${title}</h1>
  </div>
`;

// Strings are IMMUTABLE – methods return new strings
let str = 'hello';
str[0] = 'H'; // silently fails
str.toUpperCase(); // 'HELLO' – new string, str unchanged
```

10.2 String Methods Reference

Method	Example	Result / Notes
toUpperCase/Lower()	'Hello'.toUpperCase()	'HELLO'
trim/Start/End()	' hi '.trim()	'hi' — removes whitespace
split(sep)	'a,b,c'.split(',')	['a','b','c']
slice(s,e)	'hello'.slice(1,3)	'el' — supports negatives
indexOf(str)	'hello'.indexOf('l')	2 (-1 if not found)
includes(str)	'hello'.includes('ell')	true
startsWith/endsWith()	'hello'.startsWith('he')	true
replace(s,r)	'hi world'.replace('world','JS')	first match only
replaceAll(s,r)	'aababc'.replaceAll('a','x')	ES2021
padStart(n,fill)	'5'.padStart(3,'0')	'005'
padEnd(n,fill)	'hi'.padEnd(5,'.')	'hi...'
repeat(n)	'ha'.repeat(3)	'hahaha'
charAt(i) / at(i)	'hello'.at(-1)	'o' — at() supports negatives
charCodeAt(i)	'A'.charCodeAt(0)	65
match(regex)	'test123'.match(/[0-9]+)/	['123']

10.3 Interview Questions

Q: How do you reverse a string in JavaScript?

`str.split("").reverse().join("")` — split into chars, reverse array, join back. For proper Unicode (emoji): `Array.from(str).reverse().join("")` handles multi-byte characters correctly.

Q: How do you check if a string is a palindrome?

`const isPalindrome = s => s === s.split('').reverse().join('');` Or without reversing: iterate from both ends comparing `s[0]` with `s[length-1]`, `s[1]` with `s[length-2]`, etc.

Q: What is the difference between slice and substring for strings?

`slice`: supports negative indices (counts from end); if `start > end`, returns `''`. `substring`: treats negatives as 0; if `start > end`, swaps them. Prefer `slice` as it's more consistent.

Objects

11.1 Creating Objects

```
// Object literal (most common)
const user = {
  name: 'Alice',
  age: 30,
  'full name': 'Alice Smith', // quoted key for special chars
};

// ES6 Class
class Animal { constructor(n) { this.name = n; } }

// Factory function
function createUser(name, age) { return { name, age }; }

// Object.create() – inherit from prototype
const proto = { greet() { return `Hi, I'm ${this.name}`; } };
const obj = Object.create(proto);
obj.name = 'Charlie';
```

11.2 Accessing & Modifying Properties

```
const obj = { name: 'Alice', age: 30 };

obj.name          // 'Alice' – dot notation
obj['age']         // 30      – bracket notation

const key = 'name';
obj[key]           // 'Alice' – dynamic key access

obj.email = 'alice@example.com'; // add property
delete obj.age;                 // remove property

// Check property existence
'name' in obj                // true (own + inherited)
obj.hasOwnProperty('name')   // true (own only)
Object.hasOwn(obj, 'name')    // ES2022 – preferred
```

11.3 Object Methods

```
const obj = { a: 1, b: 2, c: 3 };

Object.keys(obj)    // ['a','b','c'] – own enumerable keys
Object.values(obj)  // [1,2,3]       – own enumerable values
Object.entries(obj) // [['a',1],['b',2],['c',3]]

// Iterate over entries
for (const [key, val] of Object.entries(obj)) {
  console.log(`${key}: ${val}`);
}
```

```

}

// Merge objects
const merged = Object.assign({}, obj, { d: 4 });

// Make immutable
const frozen = Object.freeze({ x: 1 });
frozen.x = 99; // silently fails (TypeError in strict mode)

// fromEntries – array of pairs → object
Object.fromEntries([['a', 1], ['b', 2]]); // {a:1, b:2}

```

11.4 Destructuring

```

const user = { name: 'Alice', age: 30, role: 'admin' };

// Basic
const { name, age } = user;

// Rename
const { name: userName, age: userAge } = user;

// Default values
const { name, score = 100 } = user; // score = 100

// Rest
const { name: n, ...rest } = user; // rest = {age:30, role:'admin'}

// Nested
const { address: { city } } = { address: { city: 'London' } };

// Function parameter destructuring
function greet({ name, age = 25 }) {
  return `${name} is ${age}`;
}

// Array destructuring
const [first, second, ...rest2] = [1, 2, 3, 4, 5];
const [, , third] = [1, 2, 3]; // skip with commas

// Swap variables
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

```

11.5 Spread Operator

```

// Merge objects (last property wins)
const defaults = { theme: 'dark', lang: 'en' };
const overrides = { lang: 'fr', fontSize: 16 };
const config = { ...defaults, ...overrides };
// { theme: 'dark', lang: 'fr', fontSize: 16 }

// Shallow copy
const copy = { ...original };

```

```
// Add/override properties
const updatedUser = { ...user, age: 31, isAdmin: true };

// Spread in function calls
const nums = [3, 1, 4, 1, 5];
Math.max(...nums); // 5

// Combine arrays
const all = [...arr1, ...arr2, 'extra'];
```

11.6 Interview Questions

Q: What is the difference between `Object.assign()` and spread operator?

Both do a shallow merge. `Object.assign(target, source)` modifies target in place and returns it. Spread `{...source}` always creates a new object. They behave identically for merging: `{...a, ...b} === Object.assign({}, a, b)`. Spread is generally preferred as it's more readable and doesn't mutate.

Q: How do you deep clone an object?

- 1) `structuredClone(obj)` — modern native deep clone (Node 17+/browsers).
- 2) `JSON.parse(JSON.stringify(obj))` — works for JSON-compatible data (no functions, no circular refs).
- 3) `Lodash _.cloneDeep(obj)` — handles edge cases. Spread and `Object.assign` are SHALLOW — nested objects remain shared references.

Interview Questions — 100 Beginner + 75 Intermediate

Section A: Beginner Questions (Selected Key Q&A)

The following covers all 100 beginner questions with concise answers. These are the most commonly tested fundamentals in JavaScript interviews.

Q: Q1. What is JavaScript?

High-level, interpreted, dynamically-typed, single-threaded, prototype-based language. The only language natively supported by all browsers. Runs everywhere via Node.js, React Native, Electron, and more.

Q: Q2. What is the difference between let, var, and const?

var: function-scoped, hoisted as undefined, can be redeclared. let: block-scoped, hoisted in TDZ, cannot be redeclared. const: block-scoped, hoisted in TDZ, cannot be reassigned (object properties can change).

Q: Q3. What are the primitive data types in JavaScript?

7 primitives: number, string, boolean, null, undefined, symbol, bigint. Everything else (objects, arrays, functions) is of type object.

Q: Q4. What is the difference between null and undefined?

undefined: declared but not assigned; default return of functions; missing arguments. null: explicitly set to 'no value'. Both are falsy. null == undefined is true; null === undefined is false.

Q: Q5. What does typeof operator return?

Returns a string: 'number', 'string', 'boolean', 'undefined', 'symbol', 'bigint', 'function', 'object'. Notable: typeof null is 'object' (bug), typeof function is 'function' (special case).

Q: Q6. What is NaN?

Not a Number — result of invalid math like 0/0 or parseInt('abc'). typeof NaN is 'number'. It's the only value not equal to itself. Check with Number.isNaN().

Q: Q7. What is the difference between == and ===?

== uses type coercion before comparing. === checks value AND type without coercion. Always prefer ===.

Q: Q8. What is a template literal?

String delimited by backticks supporting embedded expressions `${expr}`, multi-line strings, and tagged templates.

Q: Q9. What are falsy values in JavaScript?

false, 0, -0, 0n, "", null, undefined, NaN. All other values are truthy (including '0', [], {}).

Q: Q10. What is array destructuring?

Syntax to unpack array values: `const [a, b] = [1, 2]`. Supports defaults, skipping, rest: `const [first, , third, ...rest] = arr`.

Q: Q11. What is object destructuring?

Extracts object properties into variables: `const {name, age} = user`. Supports renaming (`{name: userName}`), defaults (`{score=0}`), rest (`{x, ...remaining}`).

Q: Q12. What does the spread operator do?

... spreads elements of an iterable. For arrays: `[...arr1, ...arr2]`. For objects: `{...obj1, ...obj2}` shallow merge. For function calls: `fn(...args)`.

Q: Q13. push vs pop?

`push(...items)`: adds to END, returns new length. `pop()`: removes last item, returns it.

Q: Q14. shift vs unshift?

`unshift(...items)`: adds to START, returns new length. `shift()`: removes first element, returns it.

Q: Q15. What does slice return?

A new array from start to end (exclusive). Non-mutating. Supports negative indices.

Q: Q16. What does splice do?

Mutates array in place. `splice(start, deleteCount, ...items)`. Removes and optionally inserts. Returns removed elements.

Q: Q17. map vs forEach?

`map`: returns new array of transformed values. `forEach`: returns undefined, side effects only. Chain `map`; don't chain `forEach`.

Q: Q18. How does filter work?

Creates new array with only elements where callback returns truthy. Does not mutate original.

Q: Q19. What does reduce do?

Accumulates elements into a single value. Takes `callback(accumulator, currentValue)` and optional initial value.

Q: Q20. How do you check if a value is an array?

`Array.isArray(value)` — the only reliable way. `typeof []` is 'object', `instanceof` can fail across iframes.

Q: Q21. What is a callback function?

A function passed as an argument to be called later. Examples: `setTimeout(callback, delay)`, `arr.map(callback)`.

Q: Q22. What is an arrow function?

Concise function syntax using `=>`. No `own`, `this`, `arguments`, `super`, or `prototype`. Cannot be used as constructors.

Q: Q23. What is the ternary operator?

`condition ? valueIfTrue : valueIfFalse`. Concise alternative to `if/else` for expressions.

Q: Q24. What is optional chaining?

`?.` safely accesses nested properties. Returns `undefined` instead of throwing if any part is `null/undefined`.

Q: Q25. What is nullish coalescing?

`??` returns the right side only if the left is `null` or `undefined`. Unlike `||`, doesn't trigger on `0` or `''`.

Q: Q26. for...in vs for...of?

for...in iterates over enumerable property KEYS. for...of iterates over VALUES of iterables. Don't use for...in on arrays.

Q: Q27. What is hoisting?

JS engine moves variable and function declarations to top of scope during creation phase. var hoisted as undefined; let/const in TDZ; function declarations fully hoisted.

Q: Q28. What is the Temporal Dead Zone?

Period between start of block scope and let/const declaration. Accessing during TDZ throws ReferenceError.

Q: Q29. What is scope?

Region where a variable is accessible. Types: global, function (var), block (let/const). JS uses lexical scoping.

Q: Q30. What is a closure?

A function that retains access to its outer scope's variables even after the outer function has returned.

Q: Q31. Function declaration vs expression?

Declaration: function foo(){} — hoisted fully. Expression: const foo = function(){} — only variable hoisted, not function.

Q: Q32. What is an IIFE?

Immediately Invoked Function Expression: (function(){})(). Creates an isolated scope and runs immediately.

Q: Q33. What does 'use strict' do?

Enables strict mode. Disallows undeclared variables, duplicate parameters, writing to read-only properties. Errors thrown instead of silent failure.

Q: Q34. What is the DOM?

Document Object Model — a tree representation of the HTML page that JavaScript can manipulate.

Q: Q35. What is JSON?

JavaScript Object Notation — text format for data. JSON.stringify(obj) converts JS to JSON. JSON.parse(str) converts JSON back to JS.

Q: Q36. What is localStorage?

Browser API for persisting key-value string data across sessions. setItem(key, val), getItem(key), removeItem(key). Max ~5MB. Synchronous.

Q: Q37. Synchronous vs asynchronous code?

Synchronous: runs line by line, each must complete before next. Asynchronous: operations can be initiated while program continues. setTimeout, fetch, promises are async.

Q: Q38. What is a Promise?

Object representing eventual completion/failure of an async operation. States: pending, fulfilled, rejected. Use .then(), .catch(), .finally().

Q: Q39. What is async/await?

Syntactic sugar over Promises. async makes a function return a Promise. await pauses until Promise resolves. Use try/catch for errors.

Q: Q40. What is the event loop?

Mechanism allowing non-blocking JS. Call stack runs sync code. Microtask queue (Promises) drains before macrotask queue (setTimeout). Event loop picks tasks when call stack is empty.

Q: Q41. How do you copy an object?

Shallow: `{...obj}` or `Object.assign({}, obj)`. Deep: `structuredClone(obj)`, `JSON.parse(JSON.stringify(obj))`, or `lodash_.cloneDeep()`.

Q: Q42. How to remove duplicates from an array?

`[...new Set(arr)]` — Set only stores unique values. Convert back to array with spread.

Q: Q43. What does Array.from do?

Creates array from array-like or iterable. `Array.from('abc') = ['a','b','c']`. `Array.from({length:3}, (_,i)=>i) = [0,1,2]`.

Q: Q44. What is a Set?

Collection of unique values. `new Set([1,1,2,3]) = Set{1,2,3}`. Has `.add()`, `.has()`, `.delete()`, `.size`. Great for deduplication.

Q: Q45. What is a Map?

Key-value collection where keys can be any type. Has `.set()`, `.get()`, `.has()`, `.delete()`, `.size`. Preserves insertion order.

Q: Q46. Map vs Object?

Map: any type as key, `.size` property, direct iteration, better performance for frequent add/delete.
Object: string/Symbol keys, no `.size`, JSON serialisable.

Q: Q47. What is short-circuit evaluation?

`&&`: stops at first falsy value. `||`: stops at first truthy value. Used for conditionals: `condition && doSomething()`.

Q: Q48. What is string immutability?

Strings cannot be changed after creation. `str[0]='X'` silently fails. All string methods return new strings.

Q: Q49. What is a pure function?

Always returns the same output for same input and has no side effects (no external state modification, no I/O).

Q: Q50. What is a side effect?

Any modification of state outside the function's scope: modifying a variable, network requests, DOM changes, `console.log`.

Q: Q51. What does Number.isInteger() do?

Returns true if value is a finite integer. `Number.isInteger(4.0)` is true (`4.0 === 4`).
`Number.isInteger(4.5)` is false.

Q: Q52. parseInt vs Number for strings?

`Number('42px') = NaN` (strict). `parseInt('42px') = 42` (stops at first non-digit). Always pass radix: `parseInt(str, 10)`.

Q: Q53. Pass-by-value vs pass-by-reference?

Primitives: passed by value — copy, changes don't affect original. Objects/arrays: passed by reference — mutations affect original. The reference itself is passed by value.

Q: Q54. What does the delete operator do?

Removes an own property from an object. Returns true. Cannot delete variables or function declarations.

Q: Q55. What does `Array.prototype.includes()` do?

Returns true/false. Uses SameValueZero — can find NaN (unlike `indexOf`). `[1,NaN].includes(NaN) = true`.

Q: Q56. What does `flatMap()` do?

Combines `map()` and `flat(1)` in one pass. `[1,2,3].flatMap(n => [n, n*2]) = [1,2,2,4,3,6]`.

Q: Q57. `for...of` vs `forEach`?

`for...of`: any iterable, supports break/continue/return/await. `forEach`: arrays only, no break, ignores return values.

Q: Q58. What does `Object.freeze()` do?

Prevents adding, deleting, or modifying properties. Neither is deep — nested objects remain mutable.

Q: Q59. What is the arguments object?

Array-like object inside regular functions containing all passed arguments. NOT available in arrow functions. Not a real array — prefer rest params.

Q: Q60. What is a tagged template literal?

`fn`template ${expr}`` — calls `fn` with `(strings, ...values)`. Used in styled-components, GraphQL, SQL.

Q: Q61. What does `Number.isNaN()` differ from `isNaN()`?

`isNaN()` coerces first: `isNaN('hello') = true`. `Number.isNaN()` is strict: `Number.isNaN('hello') = false`. Only true for actual NaN.

Q: Q62. What is `Object.entries()`?

Returns array of `[key, value]` pairs from own enumerable string-keyed properties. Useful for iteration and converting to Map.

Q: Q63. What is `Object.fromEntries()`?

Transforms `[key, value]` pairs into an object. Inverse of `Object.entries()`. `Object.fromEntries(map)` converts Map to object.

Q: Q64. How do you check if an object is empty?

`Object.keys(obj).length === 0` is the most reliable way.

Q: Q65. What is `globalThis`?

Universal way to access the global object in any environment. `window` in browser, `global` in Node.js. Introduced ES2020.

Q: Q66. What is a `WeakRef`?

Holds a weak reference to an object — doesn't prevent garbage collection. `weakRef.deref()` returns object or undefined if collected.

Q: Q67. What does `sort()` do without arguments?

Sorts in place, converting elements to strings and sorting lexicographically. `[10,1,21].sort()` gives `[1,10,21]`. Always provide compare function for numbers.

Q: Q68. How do you find max value in an array?

`Math.max(...arr)`. For objects: `arr.reduce((max, x) => x.val > max.val ? x : max)`.

Q: Q69. What is `String.prototype.at()`?

ES2022 method supporting negative indices. `'hello'.at(-1) = 'o'`. Cleaner than `str[str.length-1]`.

Q: Q70. What is `Array.prototype.fill()`?

Fills all (or range of) elements with a static value. `new Array(5).fill(0) = [0,0,0,0,0]`. Mutates original.

Q: Q71. What does `flat()` do?

`[1,[2,[3]]].flat() = [1,2,[3]]` (depth 1). `[1,[2,[3]]].flat(Infinity) = [1,2,3]`. Non-mutating.

Q: Q72. What is computed property syntax?

`const key='name'; const obj={[key]:'Alice'}` — property name computed at runtime.

Q: Q73. What is optional catch binding (ES2019)?

`try { } catch { }` — you can omit the error parameter if you don't need it.

Q: Q74. What is string padding?

`'5'.padStart(3,'0') = '005'`. `'hi'.padEnd(5,'.') = 'hi...'`. Used for formatting and zero-padding.

Q: Q75. What is a named function expression?

`const factorial = function fact(n){...}`. The name `'fact'` is only accessible inside the function body — useful for recursion.

Q: Q76. How to convert an array to a string?

`arr.join(separator)` — default comma. `[1,2,3].join('-') = '1-2-3'`. `arr.toString()` uses comma.

Q: Q77. What is `(255).toString(16)`?

`'ff'` — converts number to string in given base. `(8).toString(2) = '1000'` (binary).

Q: Q78. What are `Symbol.iterator` and the iteration protocol?

An object is iterable if it has `[Symbol.iterator]()` returning an iterator with `next()` method returning `{value, done}`. `for...of` uses this protocol.

Q: Q79. What does `Array.prototype.some()` return?

true if at least one element passes the callback. Short-circuits on first truthy result.

Q: Q80. What is `Number.MAX_SAFE_INTEGER`?

$2^{53} - 1 = 9007199254740991$. Integers above this lose precision in JS numbers. Use `BigInt` for larger integers.

Section B: Intermediate Questions (Selected Key Q&A)

Q: IQ1. Explain shallow copy vs deep copy.

Shallow copy: copies only the top level — nested objects are still shared references. Methods: `{...obj}`, `Object.assign({}, obj)`, `arr.slice()`. Deep copy: recursively copies all levels. Methods: `structuredClone(obj)` [modern], `JSON.parse(JSON.stringify(obj))` [JSON-compatible], `lodash _cloneDeep()`.

Q: IQ2. What is the output of `0.1 + 0.2` and why?

`0.30000000000000004`. JavaScript uses IEEE 754 double-precision floating point. `0.1` and `0.2` cannot be represented exactly in binary. Fix: `toFixed(1)`, `Math.round(result * 10) / 10`, or work in integers (e.g. cents).

Q: IQ3. What is the event loop and how does it work?

JS is single-threaded. Event loop monitors call stack and task queues. When call stack is empty: 1) Drain microtask queue completely (Promises, `queueMicrotask`). 2) Take ONE macrotask (`setTimeout`, I/O). 3) Render frame (browser). 4) Repeat. Promises always fire before `setTimeout`.

Q: IQ4. What will `console.log(typeof typeof 42)` print?

`'string'`. `typeof 42` returns the string `'number'`. `typeof 'number'` returns `'string'`. `typeof` always returns a string, so `typeof` of any `typeof` expression is always `'string'`.

Q: IQ5. What is the prototype chain?

Every JS object has `[[Prototype]]` pointing to another object. Property access walks this chain: own properties first, then `[[Prototype]]`, then `[[Prototype]]`'s `[[Prototype]]`, until null. If not found: undefined.

Q: IQ6. How does 'new' work internally?

1) Creates a new empty object. 2) Sets `[[Prototype]]` to `Constructor.prototype`. 3) Calls Constructor with `this = new object`. 4) Returns new object — unless Constructor returns a different object, which is returned instead.

Q: IQ7. What is Promise chaining?

Each `.then()` returns a new Promise. If callback returns a value, new Promise resolves with it. If it returns a Promise, chain waits for that to settle. If it throws, rejection propagates to `.catch()`. Creates a flat pipeline.

Q: IQ8. Explain closures with a practical example.

A closure retains access to its outer scope after outer function returns. function `createCounter(start=0) { let count=start; return { inc(){count++;}, get(){return count;} }; }` — `count` is private, only accessible via returned methods.

Q: IQ9. What is currying?

Transforming a function of multiple arguments into a sequence of single-argument functions. `add(1)(2)(3)` instead of `add(1,2,3)`. Enables partial application. `const curry = fn => a => b => fn(a,b)`.

Q: IQ10. What is memoisation?

Caching function results for previously seen inputs. The function returns cached result instead of recomputing. function `memoize(fn){ const cache=new Map(); return (...args)=>{ const k=JSON.stringify(args); if(!cache.has(k)) cache.set(k,fn(...args)); return cache.get(k); }; }`

Q: IQ11. Debounce vs throttle?

Debounce: executes function after N ms of silence since last call. Good for search autocomplete.

Throttle: executes at most once per N ms regardless of calls. Good for scroll/resize handlers.

Q: IQ12. What are generators?

Functions with * that can pause (yield) and resume. `function* gen(){ yield 1; yield 2; }` Generator object has `.next()` returning `{value, done}`. Used for lazy sequences, custom iterators.

Q: IQ13. Map vs WeakMap?

Map: any type as key, iterable, has `.size`, keys strongly held (prevent GC). WeakMap: object keys only, not iterable, no `.size`, keys weakly held (allow GC). Use WeakMap for private data without preventing cleanup.

Q: IQ14. How does `Array.prototype.reduce` work internally?

`reduce(callback, initialValue)` starts accumulator at `initialValue`. For each element: `accumulator = callback(accumulator, currentValue, index, array)`. Always provide initial value to avoid bugs on empty arrays.

Q: IQ15. What is `Object.defineProperty`?

Defines/modifies a property with full descriptor control: value, writable, enumerable, configurable. `Object.defineProperty(obj, 'x', {get(){...}, set(v){...}})`. Used by Vue.js 2 reactivity.

Q: IQ16. Explain the Proxy object.

Proxy wraps an object and intercepts operations. `const proxy = new Proxy(target, {get(t,k){return t[k]}})`. Useful for validation, logging, virtual properties. Vue 3 uses Proxy for reactivity.

Q: IQ17. What is async iteration?

for await...of loops over async iterables. Each iteration awaits next value. `for await (const chunk of readableStream) {}`. Requires async context.

Q: IQ18. What if you await a non-Promise?

Value is wrapped in `Promise.resolve()`. `await 42` gives 42. This is why `await` works regardless of whether a function is sync or async.

Q: IQ19. `Promise.all` vs `Promise.allSettled`?

`Promise.all`: rejects immediately on first rejection. `Promise.allSettled`: waits for ALL, returns `[{status:'fulfilled', value} | {status:'rejected', reason}]`. Use `all` when all must succeed; `allSettled` when partial failure is OK.

Q: IQ20. What is the structured clone algorithm?

`structuredClone(obj)` performs deep cloning. Handles circular references, Date, Map, Set, ArrayBuffer. Does NOT handle functions, DOM nodes, class instances. Better than `JSON.parse/stringify` for most deep copy needs.

Q: IQ21. How is `async/await` compiled internally?

`async/await` is syntactic sugar over Promises and generators. Each `await` is a yield point. The runtime wraps the generator in a Promise executor, calling `.next()` when each awaited Promise resolves.

Q: IQ22. Microtask vs macrotask?

Microtask: higher priority, runs after current operation and BEFORE next render/macrotask. Sources: `Promise.then`, `queueMicrotask`. Macrotask: scheduled at next loop iteration. Sources: `setTimeout`, `setInterval`, I/O. ALL microtasks drain before any macrotask.

Q: IQ23. Implement `Promise.all` from scratch.

```
function myAll(promises){ return new Promise((res, rej) => { const results = []; let remaining = promises.length; if (!remaining) return res([]); promises.forEach((p, i) => Promise.resolve(p).then(v => { results[i]=v; if(--remaining===0) res(results); }).catch(rej)); }); }
```

Q: IQ24. What is the iterator protocol?

An object is an iterator if it has a `next()` method returning `{value, done}`. An object is iterable if `[Symbol.iterator]()` returns an iterator. `for...of`, `spread`, `destructuring` all consume iterables this way.

Q: IQ25. What is AbortController?

Cancels async operations. `const ctrl = new AbortController(); fetch(url, {signal: ctrl.signal}); ctrl.abort()` cancels the fetch. Useful for cancelling obsolete requests in search autocomplete.

Q: IQ26. What is prototype pollution?

Attack where attacker adds properties to `Object.prototype`: `obj['__proto__']['isAdmin'] = true` — affects ALL objects. Prevention: validate input keys (reject `__proto__`, `constructor`, `prototype`), use `Object.create(null)` or `Map` for user data.

Q: IQ27. Object.keys vs Object.getOwnPropertyNames vs Reflect.ownKeys?

`Object.keys`: enumerable own string keys. `Object.getOwnPropertyNames`: all own string keys including non-enumerable. `Reflect.ownKeys`: all own keys including Symbols.

Q: IQ28. How does JavaScript handle integer overflow?

Numbers above `Number.MAX_SAFE_INTEGER` ($2^{53}-1$) lose precision. `9007199254740992 === 9007199254740993` is true! Use `BigInt` for precise large integer arithmetic: `9007199254740992n + 1n = 9007199254740993n`.

Q: IQ29. What is tail call optimisation?

When a function's last operation is a recursive call, the engine can reuse the current stack frame instead of creating a new one, preventing stack overflow. ES6 specified TCO but most engines only implement it in strict mode.

Q: IQ30. Enumerable, configurable, writable descriptors?

`enumerable`: shows in `for...in`, `Object.keys`, `JSON.stringify`. `configurable`: can descriptor be changed; can property be deleted. `writable`: can value be changed. `Object.defineProperty` gives full control over all three.

Cheat Sheet & Quick Revision

Variables

var → function scope · hoisted (undefined) · can redeclare · attaches to window
 let → block scope · hoisted (TDZ) · no redeclare · use for mutable values
 const → block scope · hoisted (TDZ) · no redeclare · no reassign (object contents CAN change)

Rule: const > let > var (default to const)

Data Types

Type	Example	typeof	Notes
Number	42, 3.14, NaN	'number'	IEEE 754 float; NaN is type number
String	'hi', `\${x}`	'string'	Immutable; Unicode UTF-16
Boolean	true, false	'boolean'	Falsy: false, 0, "", null, undef, NaN
Null	null	'object'	BUG — check with === null
Undefined	undefined	'undefined'	Unassigned variables
Symbol	Symbol('id')	'symbol'	Unique; not auto-enumerable
BigInt	42n	'bigint'	No precision loss for large ints
Object	{}, [], fn(){}	'object'	By reference; Array.isArray()

Operators Quick Reference

```
// EQUALITY
=== strict (always use)      == loose (avoid)
!== strict not equal        != loose not equal

// LOGICAL (return actual values, not just true/false)
&& first falsy OR last value  || first truthy OR last value
?? first non-null/undefined    ! negation

// MODERN (ES2020+)
?. optional chaining – safe access through null/undefined
??= assign if null/undefined  ||= assign if falsy

// TYPE COERCION TRAPS
0 == false      → true    '' == false      → true
[] == false     → true    null == undefined → true
NaN == NaN     → false
null >= 0      → true    BUT null > 0 → false AND null == 0 → false
```

Scope & Hoisting

SCOPE:

- Global → accessible everywhere
- Function → var lives here
- Block → let/const live here (any {})

HOISTING:

function declaration → fully hoisted (call before definition: OK)
var → hoisted as undefined (use before: undefined)
let / const → hoisted in TDZ (use before: ReferenceError)
function expression → only variable hoisted (let=TDZ, var=undefined)

Common Interview Traps

Code	Output	Why
typeof null	'object'	Historical bug
0.1 + 0.2 === 0.3	false	IEEE 754 floating point
[] == false	true	[] → "" → 0 === 0
![]	false	[] is truthy
null >= 0	true	null → 0 in comparison
null == 0	false	null only equals null/undefined
NaN === NaN	false	NaN is unique
'1' + 2 + 3	'123'	+ coerces to string
1 + 2 + '3'	'33'	Left-to-right: 3 + '3' = '33'
typeof typeof 42	'string'	typeof always returns string
[10,2].sort()	[10,2]	Lexicographic: '1' < '2'
new Array(3)	sparse[3]	3 empty slots, NOT [3]
[...Array(3).keys()]	[0,1,2]	Generate 0..n-1 range

Truthy & Falsy Reference

```
// FALSY (7 values – memorise these!)
false, 0, -0, 0n, '', null, undefined, NaN

// TRUTHY TRAPS – these are ALL truthy:
'0' ' ' [] {} -1 Infinity function(){} new Date()

// Boolean conversion:
!!value // double negation to boolean
Boolean(val) // explicit conversion
```

Modern JavaScript (ES6+) Quick Reference

Feature	Syntax / Example
Destructuring	const {a,b} = obj; const [x,y] = arr;
Spread	const copy = {...obj}; const arr2 = [...arr];
Rest	function fn(...args) {} const {a, ...rest} = obj;
Arrow functions	const fn = (x) => x * 2;
Template literals	`Hello \${name}!` – multi-line, expressions
Default params	function fn(x, y = 10) {}
Optional chaining	obj?.prop?.nested arr?.[0] fn?.()
Nullish coalescing	x ?? 'default' – only null/undefined triggers

Logical assignment	<code>x ??= 5; x = 5; x &&= 5;</code>
Classes	<code>class Animal { #private = 0; get count(){}} }</code>
Modules	<code>import {fn} from './mod.js'; export const x = 1;</code>
Promise.all	<code>await Promise.all([p1, p2]);</code>
async/await	<code>async function fn() { const x = await fetch(url); }</code>
structuredClone	<code>structuredClone(obj)</code> – native deep clone
BigInt	<code>const big = 9007199254740993n;</code>
Symbol	<code>const id = Symbol('unique key');</code>
Array from range	<code>[...Array(5).keys()]</code> → <code>[0,1,2,3,4]</code>
Remove duplicates	<code>[...new Set(arr)]</code>
Object from entries	<code>Object.fromEntries(entries)</code>

End of Part 1 — Fundamentals & Core Language

Continue with Part 2: Advanced JavaScript & Internals